Before start, make sure you've read the following materials:

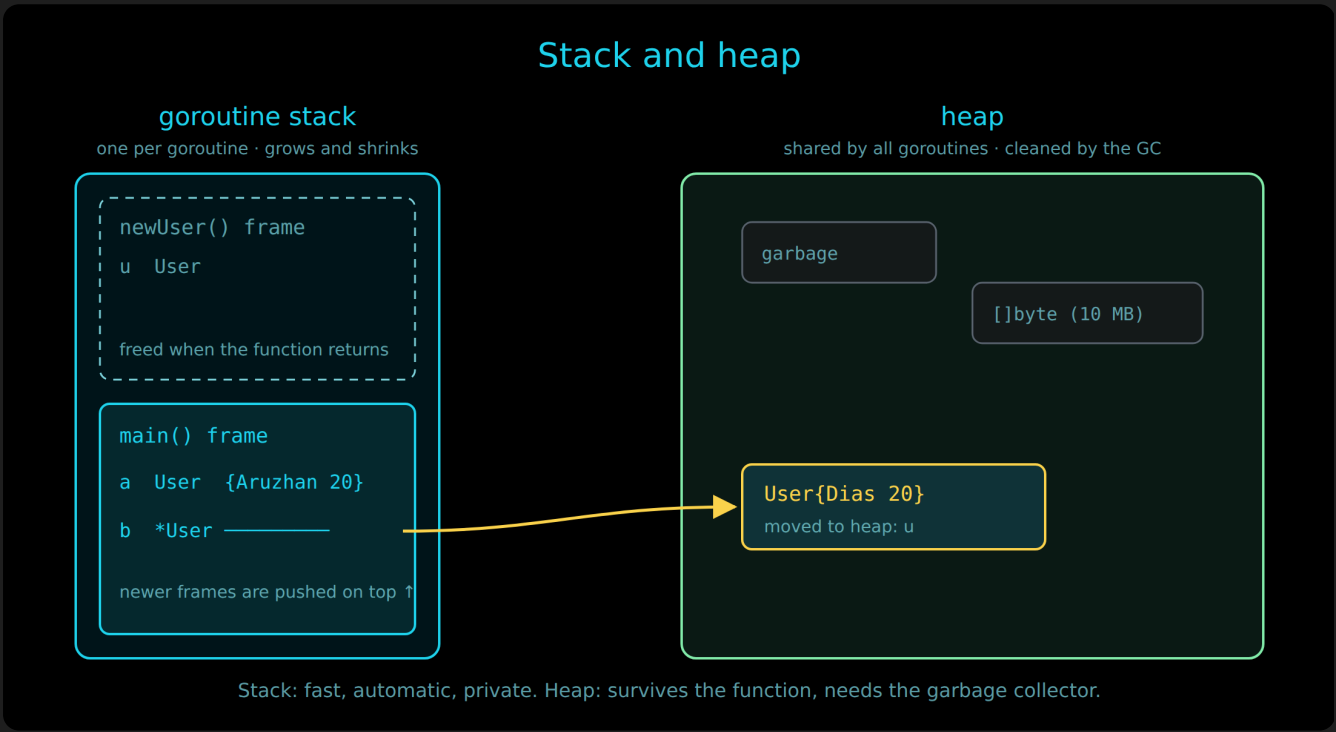
- Go FAQ: How do I know whether a variable is allocated on the heap or the stack?
- A Guide to the Go Garbage Collector (read up to "Memory limit")
- Go Blog: The Green Tea Garbage Collector
- Language mechanics on stacks and pointers
- Language mechanics on escape analysis
- Memory optimization and GC tuning in Go: GOGC and GOMEMLIMIT
- Understanding the Go garbage collector: simply and with a gopher
- Language mechanics on memory profiling (optional)

> **Note**
>
> All outputs in this tutorial were produced with **Go 1.24**. Newer compilers (Go 1.25–1.27) keep more slices on the stack, and since **Go 1.26** the default garbage collector is the new **Green Tea** collector. So your `-gcflags=-m` lines and numbers may be slightly different. That is expected — on the defence, explain what **you** see.

Part 1: Stack and heap

> **Important**
>
> ***Every Go program uses two kinds of memory. The stack is fast, private to one goroutine and cleaned automatically when a function returns. The heap is shared, lives as long as something points to it, and is cleaned by the garbage collector (GC).***

	Stack	Heap
Who owns it	one goroutine	the whole program
Allocation	move a pointer — almost free	find free space — costs time
Freeing	automatic, when the function returns	the garbage collector, later
Size	starts small, grows by copying (up to 1 GB on 64-bit)	as much as the machine gives
Good for	local, short-lived values	values that outlive their function or are shared



Each function call gets a frame

```
type User struct {
    Name string
    Age  int
}

func newUserValue(name string) User {
    u := User{Name: name, Age: 20}
    return u // a COPY of u goes back to the caller
}

func newUserPointer(name string) *User {
    u := User{Name: name, Age: 20}
    return &u // a pointer to u goes back...
}
```

Question

`u` is a local variable of `newUserPointer`. The function's stack frame disappears when it returns — so where does `&u` point to? In C this is a famous bug ("dangling pointer"). Why is it safe in Go?

Tip

In Go you never choose stack or heap. `new(T)`, `&T{}` and `make` do not mean "heap". The compiler decides where each value lives. The language spec doesn't even mention the words "stack" and "heap".

Goroutine stacks grow

```
func depth(n int) int {
    var pad [64]byte // make every frame a bit bigger
    pad[n%len(pad)] = byte(n)
    if n == 0 {
        return 0
    }
    return depth(n-1) + 1 + int(pad[(n+1)%len(pad)]) // reads a 0, but the compiler can't
know that
}

func main() {
    fmt.Println("depth:", depth(1_000_000)) // works!
}
```

A goroutine starts with a tiny stack (a few KB). When it runs out of space, the runtime allocates a bigger stack and copies the old one into it. That's why one million nested calls work. But there is a limit:

```
func forever(n int) int {
    return forever(n+1) + 1
}
```

```
runtime: goroutine stack exceeds 1000000000-byte limit
fatal error: stack overflow
```

> **Task**
>
> **Task 1: Explore the stack**
>
> 1. Run `depth(1_000_000)`. Then increase the size of `pad` to `[1024]byte`. Does it still work? Estimate how much stack memory one call uses in each case. (You can see the real frame size with `go build -gcflags=-S`: look for `TEXT main.depth(SB), ABIInternal, $NN-8`.)
>
> 2. Remove the line that reads `pad` and check the frame size again. What did the compiler do?
>
> 3. Run `forever(0)` and read the error. Is it a `panic` that `recover` can catch (Week 6)? Try it.
>
> **Participation Task:** explain on the defence why the goroutine stack needs to be copied when it grows, and why that is safe even if some pointers point into the stack.

Part 2: Escape analysis

Escape analysis is the part of the compiler that decides: can this value stay in the function's stack frame, or might it be used **after** the function returns? If it might, the value **escapes** to the heap.

How to see the compiler's decisions

```
go build -gcflags='-m' .        # short report
go build -gcflags='-m -l' .     # -l disables inlining: easier to read
go build -gcflags='-m -m -l' .  # also explains WHY
```

> **Note**
>
> Inlining copies small functions into their callers, and that can change escape decisions. For learning, `-l` makes the output match the code you wrote. Real builds have inlining enabled.

The examples

```go
func newUserValue(name string) User {        // 1: return a value
    u := User{Name: name, Age: 20}
    return u
}

func newUserPointer(name string) *User {     // 2: return a pointer
    u := User{Name: name, Age: 20}
    return &u
}

func birthday(u *User) {                     // 3: pass a pointer down
    u.Age++
}

func counter() func() int {                  // 4: closure (Week 6)
    count := 0
    return func() int {
        count++
        return count
    }
}

var registry []*User

func register(u *User) {                     // 5: store a pointer globally
    registry = append(registry, u)
}

func smallArray() int  { buf := make([]byte, 1024); return len(buf) }    // 6
func bigArray() int    { buf := make([]byte, 10<<20); return len(buf) }  // 7: 10 MB
func dynamicSize(n int) int { buf := make([]byte, n); return len(buf) }  // 8
```

`go build -gcflags='-m -l'`:

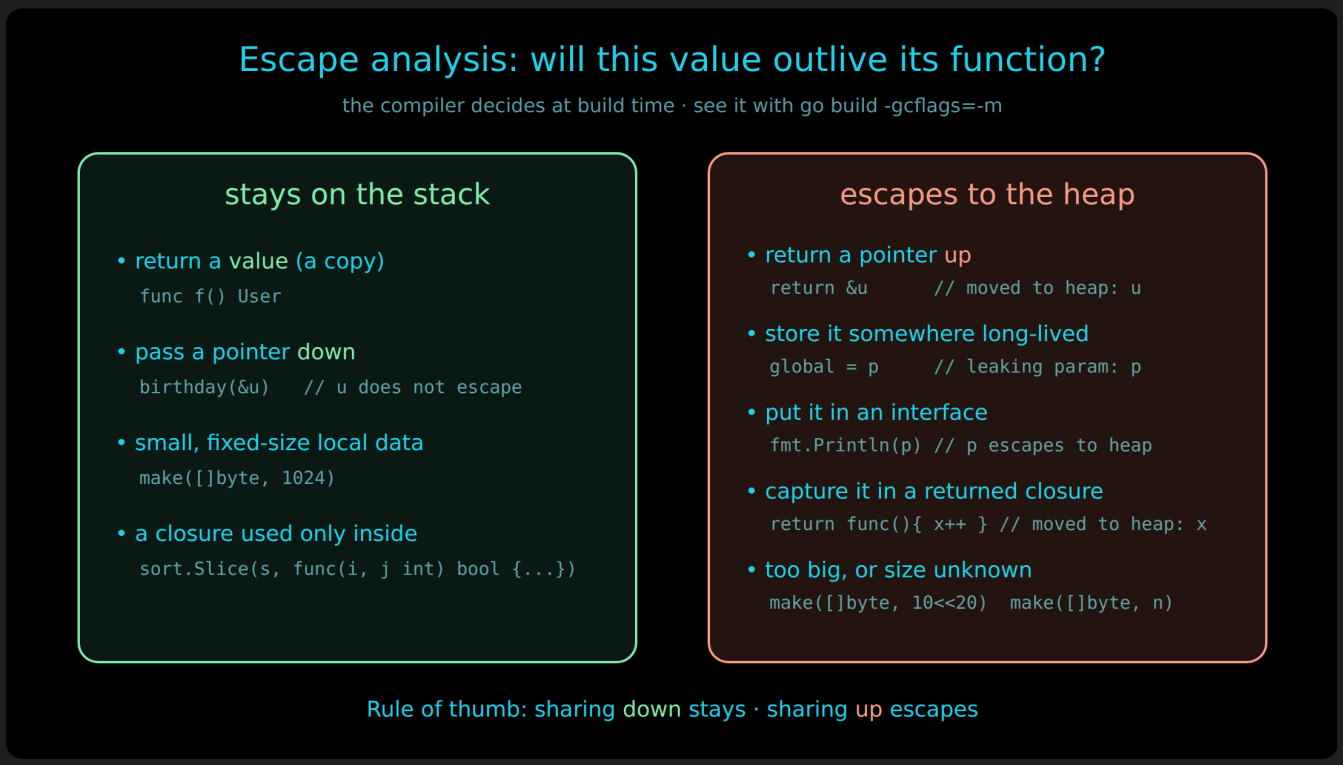
```
# (line numbers removed; the function each line belongs to is shown on the right)
leaking param: name                          # 2 newUserPointer
moved to heap: u                             # 2 newUserPointer
u does not escape                            # 3 birthday
moved to heap: count                         # 4 counter
func literal escapes to heap                 # 4 counter
leaking param: u                             # 5 register
make([]byte, 1024) does not escape           # 6 smallArray
make([]byte, 10485760) escapes to heap       # 7 bigArray
make([]byte, n) escapes to heap              # 8 dynamicSize
```

How to read the report

Message	Meaning
`moved to heap: u`	the variable `u` itself lives on the heap
`... escapes to heap`	this value (a `make`, a literal, a value boxed into an interface) is allocated on the heap
`does not escape`	stays on the stack
`leaking param: p`	the pointer `p` is stored or returned, so whatever the **caller** passes may have to live on the heap
no line at all	nothing to report — e.g. `newUserValue` returns a copy, `u` stays on the stack

Ask for the reason with `-m -m`:

```
u escapes to heap:
  flow: ~r0 = &u:
    from &u (address-of)
    from return &u (return)
...
make([]byte, 10485760) escapes to heap:
  flow: {heap} = &{storage for make([]byte, 10485760)}:
    from make([]byte, 10485760) (too large for stack)
```



What makes a value escape?

- **Sharing up:** returning a pointer to a local (`return &u`).
- **Storing a pointer somewhere that lives longer:** a global, a map, a slice that escapes, a channel.
- **Interfaces:** passing a value as `any` / `interface` usually boxes it on the heap — `fmt.Println(x)` makes `x` escape (Week 5, boxing).
- **Closures** that outlive the function capture their variables on the heap (Week 6).
- **Size:** too big for the stack (in Go 1.24: `make` > 64 KB, local variables > 128 KB), or size not known at compile time (`make([]byte, n)`).

> ⚠ **Warning**
>
> The size limits and the `make([]byte, n)` rule depend on the compiler version. Go 1.25 and 1.26 learned to keep small slices with a non-constant size on the stack in more cases. Never write code that **relies** on a value being on the stack — measure instead.

> **Task**
>
> ### Task 2: Predict, then check
>
> For each function, write down **before running** whether the marked value stays on the stack or escapes, and why. Then run `go build -gcflags='-m -l'` and compare. Explain every mismatch.

```
type Point struct{ X, Y int }

var global *Point

func f1() int          { x := 42; return x }
func f2() *int         { x := 42; return &x }
func f3(p *Point) int  { return p.X + p.Y }
func f4()              { p := Point{1, 2}; fmt.Println(p) }
func f5() []int        { s := make([]int, 10); return s }
func f6() int          { s := make([]int, 10); return len(s) }
func f7(n int) int     { s := make([]int, n); return len(s) }
func f8() func() int   { x := 0; return func() int { x++; return x } }
func f9(p *Point)      { global = p }
func f10() byte        { var big [200_000]byte; return big[0] }

func main() {
    p := Point{3, 4}
    f9(&p) // what happens to p in main?
}
```

Part 3: Allocations

> **Important**
>
> ***Every heap allocation costs twice: once when memory is allocated, and again later when the garbage collector has to find and free it. Fewer allocations → less GC work → faster programs.***

How to count allocations

1. In a benchmark — `-benchmem` (you used it in Weeks 3, 5 and 6):

```
go test -bench=. -benchmem
```

2. In a test — `testing.AllocsPerRun` returns the average number of allocations of a function:

```
func TestJoinIDsAllocs(t *testing.T) {
    allocs := testing.AllocsPerRun(1000, func() {
        sinkS = JoinIDsFast(ids)
    })
    if allocs > 1 {
        t.Errorf("JoinIDsFast: %v allocs, want at most 1", allocs)
    }
}
```

3. In a running program — `runtime.ReadMemStats`:

```
var m runtime.MemStats
runtime.ReadMemStats(&m)
fmt.Println("heap in use:", m.HeapAlloc>>20, "MB")
fmt.Println("allocations:", m.Mallocs)
fmt.Println("GC cycles:  ", m.NumGC)
```

Typical sources of allocations — and fixes

Pattern	Why it allocates	Fix
`return &Point{x, y}` for a small struct	the struct escapes	return a value: `Point{x, y}`
`s += x` in a loop (Week 3)	a new string every time	`strings.Builder` + `Grow`
`append` without capacity (Week 2)	the array is reallocated as it grows	`make([]T, 0, n)`
`fmt.Sprint(n)` / passing to `any` (Week 5)	boxing + formatting	`strconv.AppendInt`, concrete types
a new buffer for every request	a fresh allocation each call	reuse it, e.g. `sync.Pool`
`[]byte` ↔ `string` conversions (Week 3)	each conversion copies	stay in one type (`bytes` package)

Real numbers

```go
type Point struct{ X, Y int }

func NewPointValue(x, y int) Point    { return Point{x, y} }
func NewPointPointer(x, y int) *Point { return &Point{x, y} }

// "1,2,3": the naive way
func JoinIDsSlow(ids []int) string {
    s := ""
    for i, id := range ids {
        if i > 0 {
            s += ","
        }
        s += fmt.Sprint(id)
    }
    return s
}

// the same result, one buffer
func JoinIDsFast(ids []int) string {
    var sb strings.Builder
    sb.Grow(len(ids) * 8)
    var tmp [20]byte
    for i, id := range ids {
        if i > 0 {
            sb.WriteByte(',')
        }
        sb.Write(strconv.AppendInt(tmp[:0], int64(id), 10))
    }
    return sb.String()
}
```

`testing.AllocsPerRun`, 10 IDs and 1000 appended numbers:

```
NewPointValue          0 allocs/op
NewPointPointer        1 allocs/op
JoinIDsSlow            31 allocs/op
JoinIDsFast            1 allocs/op
CollectNoCap(1000)     12 allocs/op
CollectWithCap(1000)   1 allocs/op
```

Question

`JoinIDsFast` still makes **one** allocation. Which one, and why can't it be avoided? Why does `CollectNoCap(1000)` make exactly **12**? (Hint: Week 2, how `cap` grows.)

Caution

Don't optimize blindly

Returning a value instead of a pointer is great for a small struct. For a big struct, copying can cost more than the allocation (remember the Week 4 benchmark). Always **measure** with a benchmark before and after.

Task

Task 3: Remove the allocations

1. Copy `JoinIDsSlow` and write a test with `testing.AllocsPerRun` that prints its allocations for 10 IDs.
2. Write `JoinIDsFast` and a test that **fails** if it makes more than 1 allocation.
3. Run `go build -gcflags='-m -l'` on both. Which line in `JoinIDsSlow` makes `id` escape?
4. Write `BenchmarkJoinIDsSlow` and `BenchmarkJoinIDsFast` and compare `ns/op`, `B/op`, `allocs/op`.

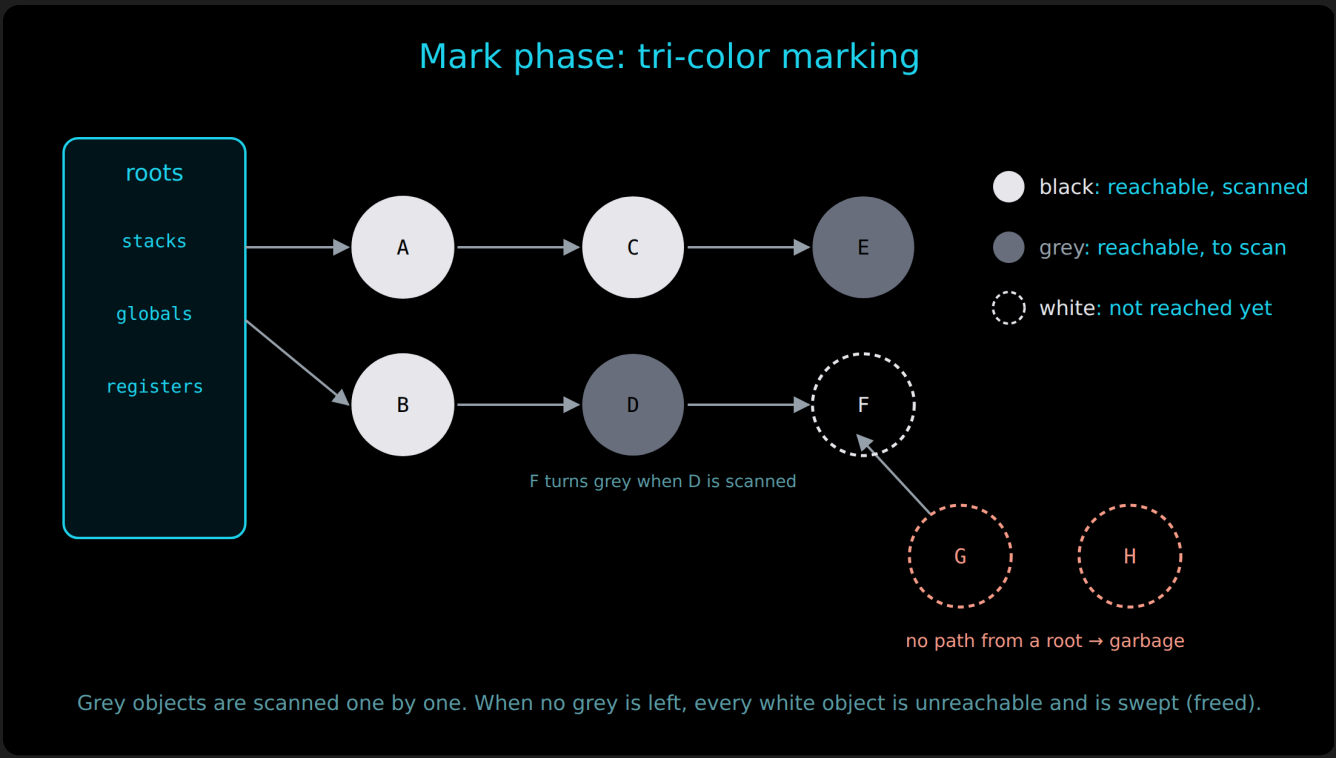
> **Task**
>
> **Task 4: Benchmark — value vs pointer, with vs without capacity**
>
> **Requirements:**
>
> - Benchmark `NewPointValue` vs `NewPointPointer` (mark both with `//go:noinline` — why?).
> - Benchmark `CollectNoCap(1000)` vs `CollectWithCap(1000)`.
> - Fill a table with `ns/op`, `B/op`, `allocs/op` and connect every result with a line from the `-gcflags=-m` report.
> - `CollectNoCap` allocates about 25 KB per call, but 1000 `int`s need only 8 KB. Where does the rest come from?

Part 4: Garbage collection basics

> **Important**
>
> ***The garbage collector frees heap memory that is no longer reachable. A value is reachable if you can get to it from a root — a goroutine's stack, a global variable — by following pointers. Everything else is garbage.***

How Go's GC works (the big picture)

- **Mark and sweep.** First the GC marks every reachable object, then it sweeps — frees everything that wasn't marked.
- **Concurrent.** Most of the work runs at the same time as your program, on other CPU cores. Your goroutines are stopped only for very short "stop-the-world" pauses (usually well under a millisecond).
- **Non-moving.** Objects stay at the same address — the GC doesn't compact the heap.
- **Write barrier.** While marking is in progress, your program keeps changing pointers. A small piece of code (the write barrier) runs on every pointer write so the GC doesn't miss a reachable object.



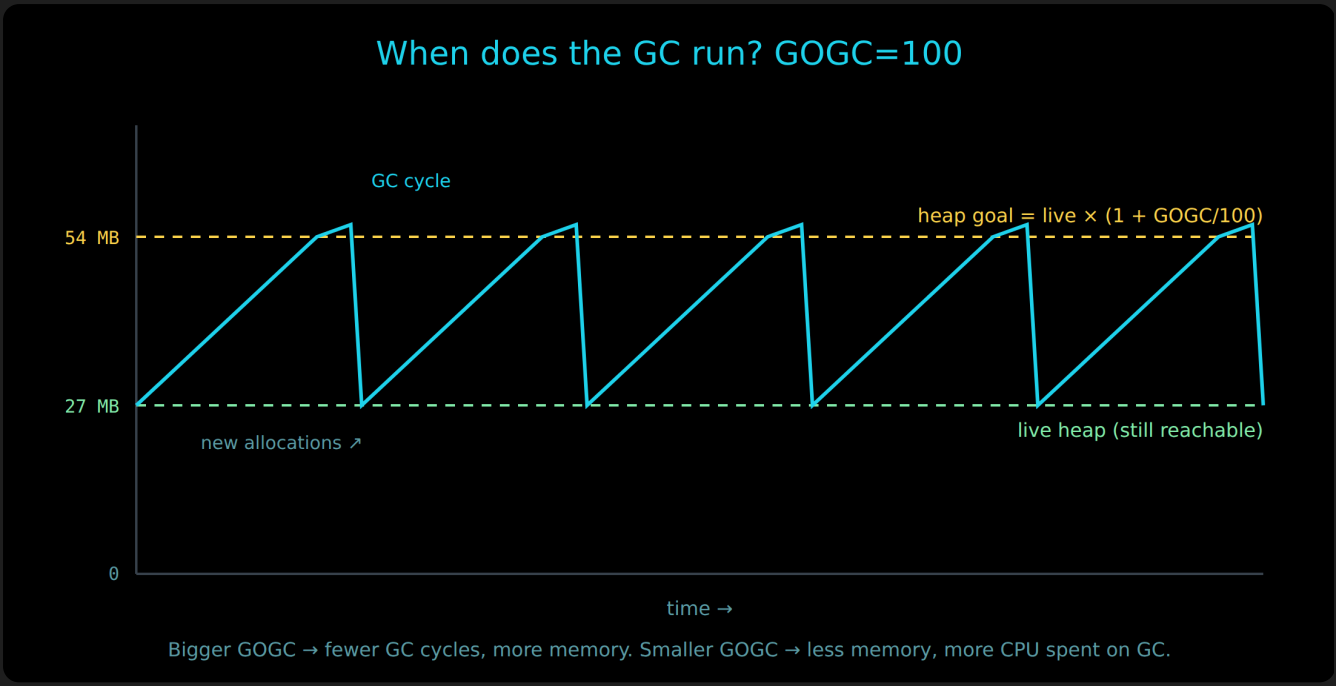
> **Note**
>
> **Green Tea (Go 1.26+)**
>
> **The new default collector still uses the same ideas — reachability, mark and sweep, GOGC. What changed is how marking walks memory: it works on whole pages of small objects instead of jumping from object to object, which is friendlier to CPU caches. The Go team reports 10–40 % less GC overhead in GC-heavy programs.**

When does the GC run? `GOGC`

The GC starts a new cycle when the heap reaches the heap goal:

```
heap goal ≈ live heap × (1 + GOGC/100)
```

With the default `GOGC=100`, the GC runs when the heap has doubled since the last collection.



Watch the GC work: `GODEBUG=gctrace=1`

```
GODEBUG=gctrace=1 ./gcdemo
```

```
gc 61 @0.453s 6%: 0.012+1.9+0.048 ms clock, ..., 46->51->27 MB, 54 MB goal, 0 MB stacks, 0 MB globals, 2 P
```

Part	Meaning
`gc 61`	the 61st GC cycle
`@0.453s`	time since the program started
`6%`	share of CPU time spent in GC so far
`0.012+1.9+0.048 ms clock`	stop-the-world pause + concurrent mark + stop-the-world pause
`46->51->27 MB`	heap at the start of GC → at the end of marking → **live** heap after
`54 MB goal`	the next heap goal: 27 MB live × 2 (GOGC=100)
`2 P`	number of processors used

The trade-off: memory vs CPU

The same program, ~22 MB of live data, ~2.7 GB of short-lived garbage:

```
type Record struct {
    ID   int
    Data [256]byte
}

var live []*Record // stays reachable: the live heap

func main() {
    for i := 0; i < 80_000; i++ {
        live = append(live, &Record{ID: i})
    }
    for i := 0; i < 10_000_000; i++ {
        r := &Record{ID: i} // garbage after one use
        sum += consume(r)
    }
    // ... print runtime.MemStats
}
```

GOGC	GC cycles	heap memory from the OS	total GC pauses	time
50	191	79 MB	11.5 ms	1.17 s
100 (default)	113	67 MB	7.8 ms	0.82 s
200	61	87 MB	4.4 ms	0.77 s
400	28	175 MB	2.1 ms	0.91 s
`off`	0	2819 MB	0	2.60 s
`off` + `GOMEMLIMIT=100MiB`	50	91 MB	4.0 ms	0.80 s

Question

With `GOGC=off` there is no GC at all — so why is it the slowest run? And why does `GOMEMLIMIT=100MiB` together with `GOGC=off` work so well here?

Controlling the GC

Setting	What it does
`GOGC=100` (env) / `debug.SetGCPercent(100)`	how much the heap may grow before the next cycle
`GOGC=off`	no GC by heap growth (dangerous on its own!)
`GOMEMLIMIT=500MiB` / `debug.SetMemoryLimit(...)`	a **soft** limit: when the total memory gets close to it, the GC runs more often (Go 1.19+)
`runtime.GC()`	forces a full collection right now — useful in tests and demos, rarely in real code

> **Tip**
>
> **In containers (Docker, Kubernetes) set `GOMEMLIMIT` a bit below the container's memory limit (e.g. 90 %). Then the GC works harder before the container gets killed for using too much memory.**

"The GC frees it for me" — not always

The GC frees only unreachable memory. If you keep a pointer to something big, it stays — even if you only need a tiny part of it. This is the memory leak promised in Week 3:

```go
func readFile(i int) []byte {
    data := make([]byte, 1<<20) // pretend we read a 1 MB file
    copy(data, fmt.Sprintf("HEADER-%04d", i))
    return data
}

func keepHeadersSlice(n int) [][]byte {
    var headers [][]byte
    for i := 0; i < n; i++ {
        data := readFile(i)
        headers = append(headers, data[:11]) // 11 bytes... or 1 MB?
    }
    return headers
}
```

```
subslices:  100 MB   # 100 headers of 11 bytes keep 100 arrays of 1 MB alive
copies:     0 MB     # bytes.Clone(data[:11])
substrings: 100 MB   # big[:5] of a 1 MB string
clones:     0 MB     # strings.Clone(big[:5])
```

Question

A slice `data[:11]` has `len` 11 — why does it keep the whole 1 MB array alive? (Week 2: what's inside a slice header?)

Task

Task 5: Find the leak

1. Write `keepHeadersSlice` and `keepHeadersCopy` (using `bytes.Clone`) and measure the live heap of each with `runtime.GC()` + `runtime.ReadMemStats`.
2. Do the same with substrings: `big[:5]` vs `strings.Clone(big[:5])`.
3. Your measurement shows `0 MB` for the leaking version? Check whether the slice is still used **after** the measurement — the GC frees variables that are not used anymore. Use `runtime.KeepAlive(x)` after the measurement.

Task

Task 6: Watch the GC

1. Build the `Record` program above and run it with `GOGC=50`, `100`, `200`, `400` and `off`. Fill the table yourself.
2. Run it with `GODEBUG=gctrace=1` and explain one line of the output, field by field.
3. Run it with `GOGC=off GOMEMLIMIT=100MiB`. How many GC cycles happen now? Why?
4. Which `GOGC` would you choose for a service with little memory? For a batch job on a big server?

Task

Task 7: Reuse buffers with `sync.Pool`

`sync.Pool` keeps objects that can be reused, so you don't allocate a new one every time. The GC may empty the pool at any time, so it is a cache, not storage.

```
var bufPool = sync.Pool{
    New: func() any { b := new(bytes.Buffer); b.Grow(4096); return b },
}

func RenderPool(ids []int) int {
    buf := bufPool.Get().(*bytes.Buffer)
    buf.Reset()
    // ... write into buf
    n := buf.Len()
    bufPool.Put(buf)
    return n
}
```

Requirements:

- **Write `RenderNew` (a new 4 KB buffer every call) and `RenderPool`. Each writes `id=<n>\n` for 200 IDs.**
- **Benchmark both with `-benchmem`. How many allocations does each make?**
- **Why must you call `buf.Reset()` after `Get`? What could go wrong if you keep using `buf` after `Put`?**

⭐ Bonus question (for those who want more)

```
func sum() int {
    nums := make([]int, 1000)
    for i := range nums {
        nums[i] = i
    }
    total := 0
    for _, n := range nums {
        total += n
    }
    return total
}
```

Does `nums` escape? Change `1000` to `10000`. Now? Find the exact size where the answer changes and explain why the compiler has such a limit.

Theoretical questions for the defence (in-person with Bakytzhan-agai):

- What is the difference between the stack and the heap? Which one is faster and why?
- Who decides whether a variable is on the stack or on the heap? Does `new` or `&` decide it?
- What is escape analysis? Name four reasons why a value escapes.
- How do you see escape analysis results? What does `leaking param` mean?
- How do you count allocations in a test and in a benchmark?
- How does Go's garbage collector decide what to free? What are roots?
- What do `GOGC` and `GOMEMLIMIT` control?
- Why can a small slice or substring keep a lot of memory alive?

Key takeaways:

- The stack is per goroutine, fast and freed automatically. The heap is shared and cleaned by the GC.
- You don't choose stack or heap — the compiler's escape analysis does. See it with `-gcflags=-m`.
- Sharing **down** usually stays on the stack; sharing **up**, interfaces, closures and big or unknown sizes escape.
- Every heap allocation costs twice: when allocated and when collected. Measure with `-benchmem` and `testing.AllocsPerRun`.
- Go's GC is a concurrent mark-and-sweep collector. It frees only unreachable memory.
- `GOGC` trades memory for CPU; `GOMEMLIMIT` sets a soft memory limit.
- A small subslice or substring can keep a huge array alive — copy it with `bytes.Clone` / `strings.Clone`.

Defence will be conducted in the next practice lesson, according to your schedule. In-person with your laptops.
